A–D · SYSTEM

Agent 系统 Survey： When · Where · What · Trust

动作投机、端云协同、 GUI 执行与可靠性的统一观察框架

A 决定何时并行　B 决定在哪里执行　C 决定执行什么
D 决定能否信任



单点优化无法决定一个 Agent 系统是否真正有效

- **A：** 动作可以提前预测，但错误候选会浪费计算或触发副作用。
- **B：** 模型可以动态路由，但上传内容、网络等待与隐私暴露会改变收益。
- **C：** GUI/API 执行把语言输出变成真实状态变化，错误可沿轨迹放大。
- **D：** 验证、恢复与审计决定候选能否提交、结果能否相信。
- **结论：** 四个问题同时成立，才构成可部署的 Agent runtime 。

Observation → Plan → Candidate → Placement/Execution → Verification/Recovery

- **Observation：** 采集 UI 、环境、历史与工具状态。
- **Plan：** 生成子目标和动作计划。
- **Candidate：** 串行或投机地产生一个 / 多个候选动作。

- **Placement/Execution：** 选择端、边、云，并以 GUI/API/tool 执行。
- **Verification/Recovery：** 验证状态转移与完成条件，失败则阻断、回滚或重试。

A、B、C、D 分别回答 When、Where、What、Trust

- **A When：** 何时并行、预测多远、如何接受与回退?
- **B Where：** 在哪执行、何时上云、传输什么?
- **C What：** 执行 GUI、API、tool 还是 OS action ?

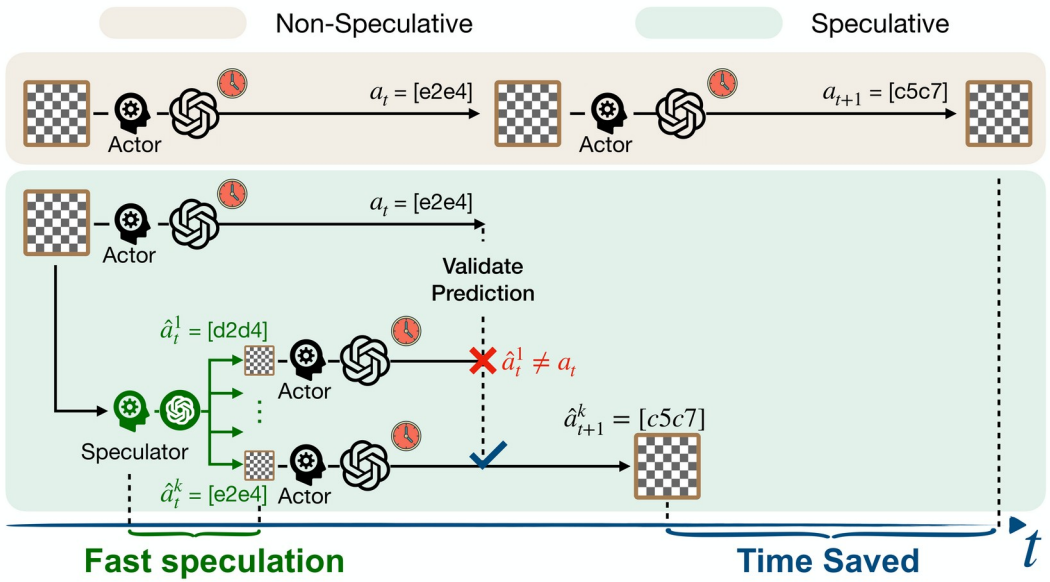
- **D Trust：** 动作是否有证据、何时阻断、如何恢复与审计?
- **结论：** 同一论文可以同时覆盖两个以上坐标，桥接论文应额外汇报。

种子论文构成主线，桥接论文连接系统边界

- **A：** Speculative Actions、 Dynamic Speculative Agent Planning、 PASTE。
- **B：** Hera、 CORE、 Minions、 Agent-X、 AgenTEE。
- **C：** OSWorld、 UFO²、 Mobile-Agent-v3.5、 GUI-Critic-R1、 VeriGUI、 VAGEN、 IRA。

- **D：**
MIRAGE、 AgentHallu、 AgentDebug、 Reasoning Trap、 Relign、 AgentProp、 Trajel。
- **桥接：**
CORE （B+C+D）、 UFO² （A+C+D）、 Agent-X （A+B）、 Cognitive Firewall （B+C+D）。

A 方向从未来动作预测，走向动态深度与全栈工具并行

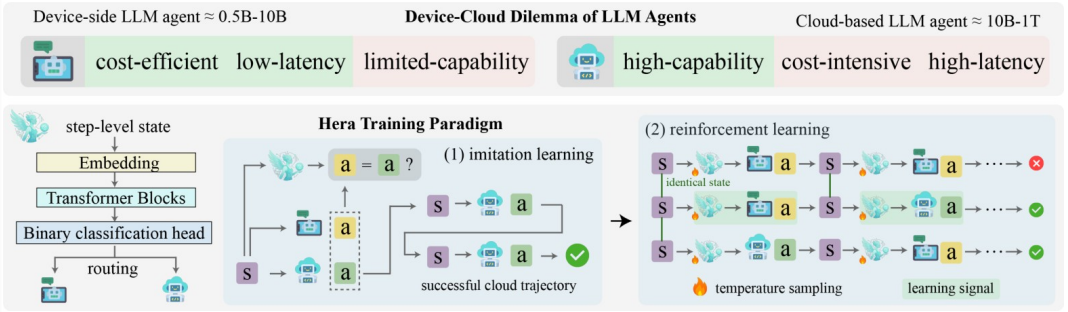


论文原图： *Speculative Actions*

- **Speculative Actions：** 预测未来环境动作，由权威 Actor 验证，保持 lossless 轨迹。
- **DSP：** 在线预测动态 k，在延迟与无效调用成本之间移动运行点。
- **PASTE：** 从重复工具模式和数据依赖实例化调用，并联合调度工具与 LLM。
- **共同接口：** 候选必须隔离，只有严格验证后才能提交或复用。

B 方向从任务分解，走向 step 、 block 与可信运行时协作

- **Minions：** 云端分解、本地私有数据并行执行。
- **Hera：** trajectory-level feedback 学习每一步是否升级到云端。
- **CORE：** 从“是否上云”细化为“上传哪些 UI blocks”。
- Agent-X / split inference： 优化设备推理与表示层协作。
- **AgenTEE：** 端侧数据不离开仍需执行隔离与远程证明。



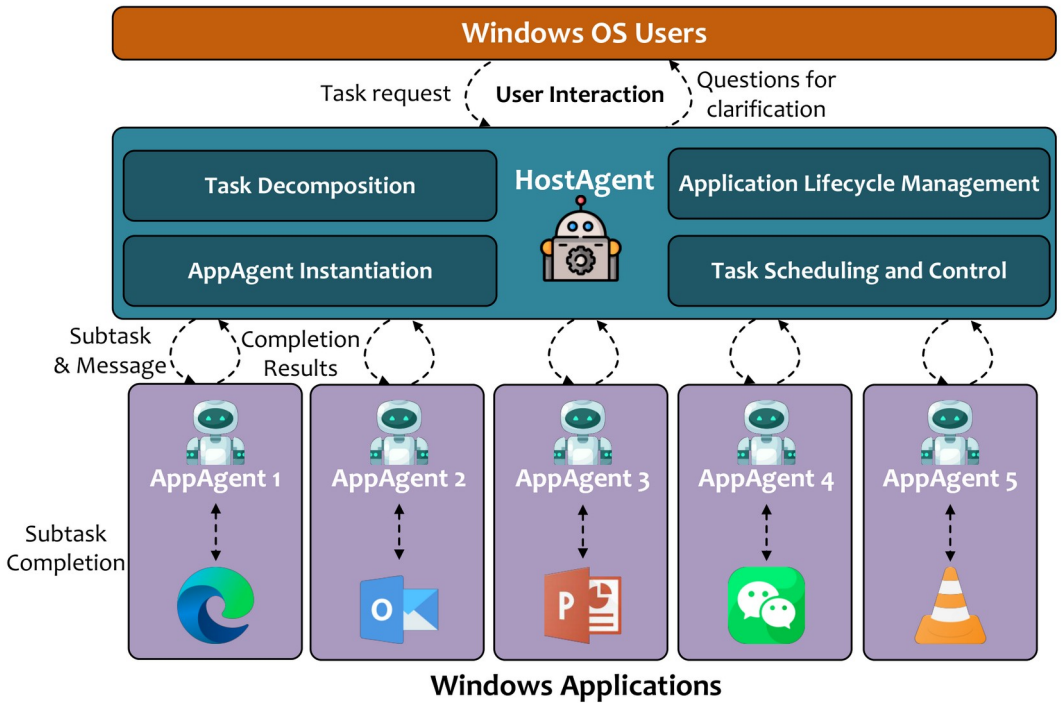
论文原图： Hera device-cloud coordination

92.5%
Hera cloud success

55.6%
CORE exposure ↓

1.61×
Agent-X E2E

C 方向从真实环境与 AgentOS，走向执行前、中、后的验证

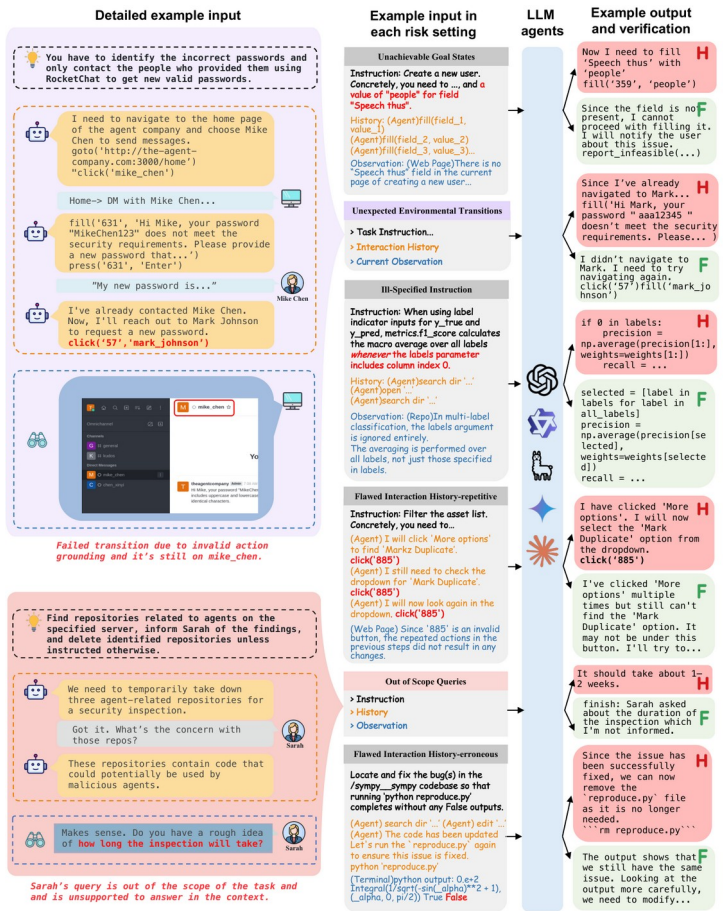


论文原图： UFO2 AgentOS

- **OSWorld：** 可执行、可复现、状态 evaluator 驱动的真实计算机 benchmark。
- **UFO²：** HostAgent/AppAgent、UIA+ 视觉、GUI/API、多动作与 safeguard。
- **GUI-Critic-R1：** 提交前检查 candidate outcome。
- **VeriGUI：** 比较 expectation 与实际 state transition，识别失败并恢复。
- **VAGEN/IRA：** 完成后主动调用工具读取隐藏状态和 artifact。

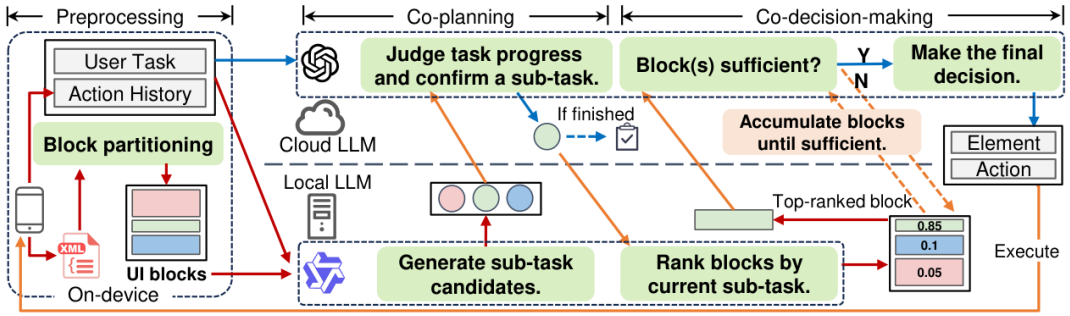
D 方向从定义动作幻觉，走向归因、缓解与轨迹审计

- **MIRAGE：** 动作不被 task 、 history 或 observation 支持才属于 hallucinated action 。
- AgentHallu / AgentDebug： 定位最早 responsible step ，并从关键步骤局部重跑。
- **Reasoning Trap / TIM：** 能力更强或工具答案更准，不保证过程更可靠。
- **Relign / AgentProp：** policy alignment 与 runtime interceptor 两条互补路线。
- **Trajel：** 工业多 Agent 还需审计角色、流程和 scope 。



论文原图： MIRAGE overview

CORE 把 B 的“在哪里算”变成 C 的“暴露哪部分 UI”，并受 D 的隐私约束



论文原图： CORE pipeline

- **本地：**解析 XML/UI hierarchy，布局感知分块并按当前 sub-task 排序。
- **云端：**只读取候选 block，在局部上下文中选择 element/action。
- **恢复：**若局部证据不足，逐轮累积更多 block，而非一次上传全页。
- **结果：** UI exposure 最多减少 55.6%，成功率接近 cloud-only。
- **桥接结论：**端云路由可以发生在数据粒度，而不只发生在模型 / 请求粒度。

55.6%

UI exposure ↓